

Universidad Tecnológica del Perú

Desarrollo Web Integrado

Guía de Laboratorio N.º 04

Implementación de API REST, herramientas de prueba y métodos HTTP

Dato	Descripción
Curso	Desarrollo Web Integrado (100000ST61)
Semana	4
Unidad	Unidad 1: API REST
Modalidad	Laboratorio práctico guiado
Herramientas	Visual Studio Code, Java 25, Maven, Spring Boot, Postman o Thunder Client
Tipo de proyecto	Back-end Spring Boot empaquetado como JAR

Propósito de la semana: Esta práctica prepara al estudiante para implementar una API REST completa antes del primer avance de proyecto. Se retoman los endpoints y controladores de la semana 2, y se incorporan métodos HTTP, respuestas con códigos adecuados, validaciones y herramientas de prueba de API.

1. Objetivo

En esta guía se desarrolla una API REST para gestionar productos. La persistencia se trabajará con base de datos en semanas posteriores; por ello, en este laboratorio se usará una estructura en memoria para concentrar la atención en REST, HTTP, controladores, respuestas, validaciones y pruebas manuales de endpoints.

2. Logro de aprendizaje

Al finalizar el laboratorio, el estudiante implementa una API RESTful en Spring Boot utilizando métodos HTTP adecuados, respuestas JSON, validación de datos y herramientas de prueba de endpoints, evidenciando el funcionamiento de operaciones CRUD sobre un recurso.

3. Producto a desarrollar

Se construirá una API REST denominada Catálogo de Productos. La API permitirá realizar las siguientes operaciones:

- Listar todos los productos con GET.
- Buscar un producto por identificador con GET.
- Filtrar productos por categoría mediante query parameters.
- Crear un producto con POST.
- Actualizar completamente un producto con PUT.
- Actualizar parcialmente el stock con PATCH.
- Eliminar un producto con DELETE.
- Responder con códigos HTTP apropiados: 200, 201, 204, 400 y 404.
- Mostrar mensajes de error en formato JSON cuando la solicitud no sea válida.

4. Conceptos teóricos necesarios

4.1. ¿Qué es una API REST?

Una API REST es una interfaz que expone recursos del sistema mediante URLs. En lugar de pensar en “pantallas”, se piensa en recursos. Por ejemplo, en esta práctica el recurso principal será producto.

Recurso: Producto
Colección de productos: /api/productos
Producto específico: /api/productos/1

REST no obliga a una tecnología específica; define una forma de organizar la comunicación cliente-servidor usando operaciones HTTP, recursos identificados por URI y representaciones como JSON.

4.2. Recurso, endpoint y URI

Concepto	Explicación	Ejemplo
Recurso	Entidad o elemento del sistema que se desea consultar o manipular.	Producto
URI	Dirección que identifica un recurso.	/api/productos/1
Endpoint	Punto de acceso publicado por la API para ejecutar una operación.	GET /api/productos
JSON	Formato textual usado para enviar y recibir datos.	{ "nombre": "Laptop" }

4.3. Métodos HTTP usados en una API REST

Método	Uso principal	Ejemplo en el laboratorio	Respuesta esperada
GET	Consultar información.	GET /api/productos	200 OK
POST	Crear un nuevo recurso.	POST /api/productos	201 Created
PUT	Reemplazar o actualizar completamente un recurso.	PUT /api/productos/1	200 OK
PATCH	Actualizar parcialmente un recurso.	PATCH /api/productos/1/stock	200 OK
DELETE	Eliminar un recurso.	DELETE /api/productos/1	204 No Content

Idea clave: El método HTTP debe expresar la intención de la operación. No es buena práctica crear, actualizar o eliminar datos usando GET, porque GET debe utilizarse para consultas.

4.4. Códigos de estado HTTP

Código	Nombre	Cuándo usarlo
200	OK	La consulta o actualización se ejecutó correctamente y se devuelve información.
201	Created	Se creó un nuevo recurso correctamente.
204	No Content	La operación fue correcta, pero no hay cuerpo de respuesta. Muy usado en DELETE.
400	Bad Request	La solicitud enviada por el cliente tiene datos inválidos o incompletos.
404	Not Found	El recurso solicitado no existe.
500	Internal Server Error	Ocurrió un error inesperado en el servidor.

4.5. Rol de Spring Boot en la API REST

Spring Boot permite construir APIs REST usando anotaciones de Spring MVC. En esta guía se usarán principalmente las siguientes:

Anotación	Función
@RestController	Indica que la clase atiende solicitudes HTTP y devuelve datos en el cuerpo de la respuesta, usualmente JSON.
@RequestMapping	Define la ruta base del controlador.
@GetMapping	Atiende solicitudes GET.
@PostMapping	Atiende solicitudes POST.
@PutMapping	Atiende solicitudes PUT.
@PatchMapping	Atiende solicitudes PATCH.
@DeleteMapping	Atiende solicitudes DELETE.
@RequestBody	Convierte el JSON recibido en un objeto Java.
@PathVariable	Lee valores que forman parte de la ruta.
@RequestParam	Lee valores enviados como parámetros de consulta.
ResponseBody	Permite controlar el código HTTP, headers y cuerpo de la respuesta.

5. Requisitos previos

- Java 25 instalado y configurado en el sistema.
- Visual Studio Code instalado.
- Extension Pack for Java instalado en VS Code.

- Spring Boot Extension Pack instalado en VS Code.
- Maven disponible desde la terminal.
- Postman o Thunder Client para probar endpoints que no son GET.
- Conocimientos previos de paquetes, clases, controladores, servicios e inyección por constructor.

6. Creación del proyecto Spring Boot

Ingrese a Spring Initializr y configure el proyecto con los siguientes valores. Si el docente ya proporcionó un proyecto base de las semanas anteriores, puede continuar sobre ese proyecto y adaptar los paquetes.

Campo	Valor recomendado
Project	Maven
Language	Java
Spring Boot	Versión estable compatible con Java 25 disponible en Spring Initializr
Group	com.utp
Artifact	semana4-api-rest
Name	semana4-api-rest
Packaging	Jar
Java	25

Agregar las dependencias:

- Spring Web: permite crear endpoints REST.
- Validation: permite usar @Valid, @NotBlank, @Positive, @Min y otras validaciones.
- Spring Boot DevTools: opcional, útil durante el desarrollo.
- Spring Boot Starter Test: útil para pruebas automáticas con JUnit, MockMvc y Mockito.

7. Abrir el proyecto en Visual Studio Code

1. Descargue el archivo ZIP generado por Spring Initializr.
2. Descomprima el proyecto en una carpeta de trabajo.
3. Abra Visual Studio Code.
4. Seleccione File > Open Folder y elija la carpeta raíz del proyecto.
5. Espere a que VS Code detecte Maven y descargue las dependencias.
6. Abra una terminal integrada con Terminal > New Terminal.

```
mvn -v
java -version
```

Ambos comandos deben responder correctamente. Si Maven o Java no se reconocen, revise la instalación antes de continuar.

8. Configuración básica de la aplicación

Abra el archivo:

```
src/main/resources/application.properties
```

Agregue o verifique la siguiente configuración:

```
spring.application.name=semana4-api-rest
server.port=8080

# Muestra detalles básicos de error durante el laboratorio.
# En producción no se recomienda exponer mensajes internos.
server.error.include-message=always
server.error.include-binding-errors=always
```

9. Estructura de paquetes recomendada

Dentro de src/main/java/com/utp/semana4 cree los siguientes paquetes:

```
com.utp.semana4
├── controller
├── dto
├── exception
├── model
└── service
```

Esta estructura separa responsabilidades: el controlador recibe solicitudes HTTP, el servicio contiene reglas de negocio, el modelo representa el recurso, los DTO controlan los datos que entran o salen y las excepciones permiten manejar errores de forma ordenada.

10. Verificar la clase principal

Verifique que la clase principal exista en el paquete base com.utp.semana4. El nombre puede variar según el proyecto generado.

```
package com.utp.semana4;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Semana4ApiRestApplication {

    public static void main(String[] args) {
        SpringApplication.run(Semana4ApiRestApplication.class, args);
    }
}
```

Importante: Los paquetes controller, service, model, dto y exception deben estar debajo del paquete base com.utp.semana4 para que Spring pueda detectarlos automáticamente.

11. Crear el modelo Producto

Cree el archivo:

```
src/main/java/com/utp/semana4/model/Producto.java
```

```
package com.utp.semana4.model;

public class Producto {

    private Long id;
    private String nombre;
    private String categoria;
    private double precio;
    private int stock;

    public Producto() {
    }

    public Producto(Long id, String nombre, String categoria, double precio, int stock) {
        this.id = id;
        this.nombre = nombre;
        this.categoria = categoria;
        this.precio = precio;
        this.stock = stock;
    }

    public Long getId() {
```

```

        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getCategoria() {
        return categoria;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }
}

```

Por ahora Producto es una clase simple de Java. No se usa @Entity porque todavía no se trabajará con base de datos. Esa parte corresponde a la unidad de back-end con bases de datos.

12. Crear DTO para registrar y actualizar productos

Cree el archivo:

```
src/main/java/com/utp/semana4/dto/ProductoRequest.java
```

```

package com.utp.semana4.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Positive;

public class ProductoRequest {

    @NotBlank(message = "El nombre es obligatorio")
    private String nombre;

    @NotBlank(message = "La categoria es obligatoria")
    private String categoria;

    @NotNull(message = "El precio es obligatorio")
    @Positive(message = "El precio debe ser mayor a cero")
    private Double precio;
}

```

```

@NotNull(message = "El stock es obligatorio")
@Min(value = 0, message = "El stock no puede ser negativo")
private Integer stock;

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getCategoria() {
    return categoria;
}

public void setCategoria(String categoria) {
    this.categoria = categoria;
}

public Double getPrecio() {
    return precio;
}

public void setPrecio(Double precio) {
    this.precio = precio;
}

public Integer getStock() {
    return stock;
}

public void setStock(Integer stock) {
    this.stock = stock;
}
}

```

12.1. ¿Por qué usamos DTO?

DTO significa Data Transfer Object. Es una clase usada para recibir o enviar datos entre cliente y servidor. Permite controlar qué campos acepta la API y aplicar validaciones sin depender directamente del modelo interno.

Error común: No todos los objetos que llegan desde el cliente deben mapearse directamente al modelo principal. En proyectos reales, usar DTO ayuda a evitar inconsistencias, datos inválidos y exposición innecesaria de campos internos.

13. Crear DTO para actualización parcial de stock

Cree el archivo:

```
src/main/java/com/utp/semana4/dto/ActualizarStockRequest.java
```

```

package com.utp.semana4.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;

public class ActualizarStockRequest {

    @NotNull(message = "El stock es obligatorio")
    @Min(value = 0, message = "El stock no puede ser negativo")
    private Integer stock;

    public Integer getStock() {
        return stock;
    }
}

```

```

    }

    public void setStock(Integer stock) {
        this.stock = stock;
    }
}

```

Este DTO se usará con PATCH porque solo se actualizará una parte del recurso: el stock.

14. Crear DTO de error

Cree el archivo:

```
src/main/java/com/utp/semana4/dto/ErrorResponse.java
```

```

package com.utp.semana4.dto;

import java.time.LocalDateTime;

public class ErrorResponse {

    private int estado;
    private String mensaje;
    private String ruta;
    private LocalDateTime fechaHora;

    public ErrorResponse(int estado, String mensaje, String ruta) {
        this.estado = estado;
        this.mensaje = mensaje;
        this.ruta = ruta;
        this.fechaHora = LocalDateTime.now();
    }

    public int getEstado() {
        return estado;
    }

    public String getMensaje() {
        return mensaje;
    }

    public String getRuta() {
        return ruta;
    }

    public LocalDateTime getFechaHora() {
        return fechaHora;
    }
}

```

Este objeto permitirá que los errores de la API también sean devueltos como JSON estructurado.

15. Crear excepción personalizada

Cree el archivo:

```
src/main/java/com/utp/semana4/exception/ProductoNoEncontradoException.java
```

```

package com.utp.semana4.exception;

public class ProductoNoEncontradoException extends RuntimeException {

    public ProductoNoEncontradoException(Long id) {
        super("No existe un producto con id: " + id);
    }
}

```

La excepción se lanzará cuando el cliente solicite, actualice o elimine un producto que no existe.

16. Crear el servicio ProductoService

Cree el archivo:

```
src/main/java/com/utp/semana4/service/ProductoService.java
```

```
package com.utp.semana4.service;

import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.stereotype.Service;

import com.utp.semana4.dto.ActualizarStockRequest;
import com.utp.semana4.dto.ProductoRequest;
import com.utp.semana4.exception.ProductoNoEncontradoException;
import com.utp.semana4.model.Producto;

@Service
public class ProductoService {

    private final Map<Long, Producto> productos = new ConcurrentHashMap<>();
    private final AtomicLong secuencia = new AtomicLong(1);

    public ProductoService() {
        registrarInicial("Laptop Lenovo", "Tecnologia", 3500.00, 10);
        registrarInicial("Mouse Logitech", "Tecnologia", 80.00, 25);
        registrarInicial("Silla ergonomica", "Muebles", 750.00, 5);
    }

    public List<Producto> listar(String categoria) {
        return productos.values()
            .stream()
            .filter(producto -> categoria == null
                || producto.getCategoria().equalsIgnoreCase(categoria))
            .sorted(Comparator.comparing(Producto::getId))
            .toList();
    }

    public Producto buscarPorId(Long id) {
        Producto producto = productos.get(id);

        if (producto == null) {
            throw new ProductoNoEncontradoException(id);
        }

        return producto;
    }

    public Producto crear(ProductoRequest request) {
        Long id = secuencia.getAndIncrement();
        Producto producto = new Producto(
            id,
            request.getNombre(),
            request.getCategoria(),
            request.getPrecio(),
            request.getStock()
        );
        productos.put(id, producto);
        return producto;
    }

    public Producto actualizar(Long id, ProductoRequest request) {
        Producto producto = buscarPorId(id);
        producto.setNombre(request.getNombre());
        producto.setCategoria(request.getCategoria());
        producto.setPrecio(request.getPrecio());
        producto.setStock(request.getStock());
        return producto;
    }
}
```

```

    }

    public Producto actualizarStock(Long id, ActualizarStockRequest request) {
        Producto producto = buscarPorId(id);
        producto.setStock(request.getStock());
        return producto;
    }

    public void eliminar(Long id) {
        Producto eliminado = productos.remove(id);

        if (eliminado == null) {
            throw new ProductoNoEncontradoException(id);
        }
    }

    private void registrarInicial(String nombre, String categoria, double precio, int stock) {
        Long id = secuencia.getAndIncrement();
        productos.put(id, new Producto(id, nombre, categoria, precio, stock));
    }
}

```

16.1. Explicación técnica del servicio

El servicio contiene la lógica de negocio de la API. En una arquitectura por capas, el controlador no debería concentrar reglas de negocio; su tarea principal es recibir la solicitud HTTP y delegar al servicio.

Se usa `ConcurrentHashMap` para mantener los productos en memoria de manera más segura ante solicitudes concurrentes. Se usa `AtomicLong` para generar identificadores incrementales sin depender de una base de datos. En semanas posteriores, este almacenamiento temporal será reemplazado por JPA, Hibernate y MySQL.

Conexión con semanas anteriores: `ProductoService` es un bean singleton de Spring. Por eso sus atributos deben manejarse con cuidado. En este laboratorio se usan estructuras concurrentes porque múltiples solicitudes HTTP podrían ejecutarse al mismo tiempo.

17. Crear el controlador REST

Cree el archivo:

```
src/main/java/com/utp/semana4/controller/ProductoController.java
```

```

package com.utp.semana4.controller;

import java.net.URI;
import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

import com.utp.semana4.dto.ActualizarStockRequest;
import com.utp.semana4.dto.ProductoRequest;
import com.utp.semana4.model.Producto;
import com.utp.semana4.service.ProductoService;

import jakarta.validation.Valid;

@RestController
@RequestMapping("/api/productos")
public class ProductoController {

```

```

private final ProductoService service;

public ProductoController(ProductoService service) {
    this.service = service;
}

@GetMapping
public ResponseEntity<List<Producto>> listar(
    @RequestParam(required = false) String categoria) {
    return ResponseEntity.ok(service.listar(categoria));
}

@GetMapping("/{id}")
public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
    return ResponseEntity.ok(service.buscarPorId(id));
}

@PostMapping
public ResponseEntity<Producto> crear(@Valid @RequestBody ProductoRequest request) {
    Producto nuevoProducto = service.crear(request);

    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest()
        .path("/{id}")
        .buildAndExpand(nuevoProducto.getId())
        .toUri();

    return ResponseEntity.created(location).body(nuevoProducto);
}

@PutMapping("/{id}")
public ResponseEntity<Producto> actualizar(
    @PathVariable Long id,
    @Valid @RequestBody ProductoRequest request) {
    return ResponseEntity.ok(service.actualizar(id, request));
}

@PatchMapping("/{id}/stock")
public ResponseEntity<Producto> actualizarStock(
    @PathVariable Long id,
    @Valid @RequestBody ActualizarStockRequest request) {
    return ResponseEntity.ok(service.actualizarStock(id, request));
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> eliminar(@PathVariable Long id) {
    service.eliminar(id);
    return ResponseEntity.noContent().build();
}
}

```

17.1. Explicación del controlador

Código	Explicación
@RestController	Convierte la clase en un controlador REST. Los objetos retornados se serializan normalmente como JSON.
@RequestMapping("/api/productos")	Define la ruta base del recurso productos.
@GetMapping	Implementa consultas.
@PostMapping	Implementa creación de productos.
@PutMapping	Implementa actualización completa.
@PatchMapping	Implementa actualización parcial.
@DeleteMapping	Implementa eliminación.
@Valid	Activa las validaciones declaradas en el DTO.
ResponseEntity	Permite devolver estado HTTP, headers y cuerpo de respuesta.

18. Crear manejador global de errores

Cree el archivo:

```
src/main/java/com/utp/semana4/exception/ApiExceptionHandler.java
```

```

package com.utp.semana4.exception;

import java.util.stream.Collectors;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import com.utp.semana4.dto.ErrorResponse;

import jakarta.servlet.http.HttpServletRequest;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(ProductoNoEncontradoException.class)
    public ResponseEntity<ErrorResponse> manejarProductoNoEncontrado(
        ProductoNoEncontradoException ex,
        HttpServletRequest request) {
        ErrorResponse error = new ErrorResponse(
            HttpStatus.NOT_FOUND.value(),
            ex.getMessage(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> manejarValidaciones(
        MethodArgumentNotValidException ex,
        HttpServletRequest request) {

        String mensaje = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(error -> error.getField() + ": " + error.getDefaultMessage())
            .collect(Collectors.joining("; "));

        ErrorResponse error = new ErrorResponse(
            HttpStatus.BAD_REQUEST.value(),
            mensaje,
            request.getRequestURI()
        );
        return ResponseEntity.badRequest().body(error);
    }
}

```

Con `@RestControllerAdvice` se centraliza el manejo de errores. Esto evita repetir try-catch en cada endpoint y permite que el cliente siempre reciba errores con una estructura JSON uniforme.

19. Ejecutar la aplicación

En la terminal integrada de VS Code, ejecute:

```
mvn spring-boot:run
```

La consola debe mostrar un mensaje similar a:

```
Tomcat started on port 8080
Started Semana4ApiRestApplication
```

Desde este momento la API estará disponible en:

```
http://localhost:8080/api/productos
```

Recordatorio importante: En el navegador no se escribe la palabra GET. Para consultar desde el navegador escriba solamente la URL: `http://localhost:8080/api/productos`. El navegador envía GET automáticamente.

20. Probar la API con el navegador

Abra el navegador y visite:

```
http://localhost:8080/api/productos
```

Debe observar una respuesta JSON con productos iniciales. Ejemplo:

```
[
  {
    "id": 1,
    "nombre": "Laptop Lenovo",
    "categoria": "Tecnologia",
    "precio": 3500.0,
    "stock": 10
  }
]
```

Luego pruebe buscar un producto específico:

```
http://localhost:8080/api/productos/1
```

Y filtrar por categoría:

```
http://localhost:8080/api/productos?categoria=Tecnologia
```

21. Probar la API con Postman o Thunder Client

Para probar POST, PUT, PATCH y DELETE se recomienda usar Postman o Thunder Client, porque el navegador común solo permite ejecutar GET de forma directa desde la barra de direcciones.

21.1. Crear producto con POST

Campo	Valor
Método	POST
URL	http://localhost:8080/api/productos
Body	raw / JSON
Content-Type	application/json

```
{
  "nombre": "Monitor Samsung",
  "categoria": "Tecnologia",
  "precio": 920.50,
  "stock": 8
}
```

Resultado esperado: código 201 Created y respuesta JSON con el producto creado. Además, la respuesta incluirá el header Location con la URL del nuevo recurso.

21.2. Actualizar producto con PUT

Campo	Valor
Método	PUT
URL	http://localhost:8080/api/productos/1
Body	raw / JSON

```
{
  "nombre": "Laptop Lenovo ThinkPad",
  "categoria": "Tecnologia",
  "precio": 3899.90,
  "stock": 12
}
```

Resultado esperado: código 200 OK y producto actualizado. PUT se usa cuando se envía una representación completa del recurso.

21.3. Actualizar parcialmente el stock con PATCH

Campo	Valor
Método	PATCH
URL	http://localhost:8080/api/productos/1/stock
Body	raw / JSON

```
{
  "stock": 20
}
```

Resultado esperado: código 200 OK y producto con stock actualizado. PATCH se usa cuando se modifica solo una parte del recurso.

21.4. Eliminar producto con DELETE

Campo	Valor
Método	DELETE
URL	http://localhost:8080/api/productos/1
Body	No enviar body

Resultado esperado: código 204 No Content. Después de eliminar, intente consultar GET /api/productos/1 y debe obtener 404 Not Found.

22. Pruebas equivalentes usando curl

curl permite probar APIs desde la terminal. En Windows se recomienda usar curl.exe para evitar conflictos con alias de PowerShell.

```
# Listar productos
curl http://localhost:8080/api/productos

# Buscar por id
curl http://localhost:8080/api/productos/1

# Crear producto
curl -X POST http://localhost:8080/api/productos ^
-H "Content-Type: application/json" ^
-d '{"nombre":"Audifonos","categoria":"Tecnologia","precio":150.0,"stock":15}'

# Actualizar stock
curl -X PATCH http://localhost:8080/api/productos/1/stock ^
-H "Content-Type: application/json" ^
-d '{"stock":30}'

# Eliminar producto
curl -X DELETE http://localhost:8080/api/productos/1
```

23. Validar errores controlados

23.1. Error 404 Not Found

Ejecute:

```
GET http://localhost:8080/api/productos/999
```

Resultado esperado:

```
{
  "estado": 404,
  "mensaje": "No existe un producto con id: 999",
  "ruta": "/api/productos/999",
}
```

```
"fechaHora": "2026-09-03T15:30:00"
}
```

23.2. Error 400 Bad Request por validación

Ejecute un POST con datos inválidos:

```
{
  "nombre": "",
  "categoria": "Tecnologia",
  "precio": -50,
  "stock": -1
}
```

Resultado esperado: código 400 Bad Request y una respuesta JSON indicando los campos inválidos.

24. Prueba automática básica con MockMvc

Como la semana anterior se trabajó TDD, se incluye una validación automática sencilla para comprobar que la API responde correctamente. Cree el archivo:

```
src/test/java/com/utp/semana4/ProductoControllerTest.java
```

```
package com.utp.semana4;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

@SpringBootTest
@AutoConfigureMockMvc
class ProductoControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void debeListarProductos() throws Exception {
        mockMvc.perform(get("/api/productos"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$[0].id").exists());
    }

    @Test
    void debeCrearProducto() throws Exception {
        String json = """
            {
                "nombre": "Tablet Xiaomi",
                "categoria": "Tecnologia",
                "precio": 1200.0,
                "stock": 7
            }
            """;

        mockMvc.perform(post("/api/productos")
            .contentType(MediaType.APPLICATION_JSON)
            .content(json)
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").exists())
            .andExpect(jsonPath("$.nombre").value("Tablet Xiaomi")));
    }
}
```

Ejecute las pruebas desde VS Code usando Run Test, desde el panel Testing o desde la terminal:

```
mvn test
```

25. Empaquetar como JAR

El curso trabaja con proyectos Spring Boot empaquetados como JAR. Para generar el artefacto ejecutable, detenga la aplicación si está corriendo y ejecute:

```
mvn clean package
```

El archivo JAR se generará dentro de la carpeta target. El nombre exacto depende del artifact definido en Spring Initializr. Para ejecutarlo:

```
java -jar target/semana4-api-rest-0.0.1-SNAPSHOT.jar
```

Luego vuelva a probar el endpoint:

```
http://localhost:8080/api/productos
```

26. Checklist de verificación funcional

N.º	Verificación	Cumple
1	El proyecto compila sin errores.	Sí / No
2	La aplicación inicia en el puerto 8080.	Sí / No
3	GET /api/productos devuelve una lista JSON.	Sí / No
4	GET /api/productos/{id} devuelve un producto existente.	Sí / No
5	POST /api/productos crea un producto y devuelve 201.	Sí / No
6	PUT /api/productos/{id} actualiza un producto completo.	Sí / No
7	PATCH /api/productos/{id}/stock actualiza solo el stock.	Sí / No
8	DELETE /api/productos/{id} devuelve 204.	Sí / No
9	Los datos inválidos devuelven 400.	Sí / No
10	Un producto inexistente devuelve 404.	Sí / No
11	mvn test ejecuta las pruebas correctamente.	Sí / No
12	mvn clean package genera el JAR.	Sí / No

27. Errores frecuentes y solución

Problema	Causa probable	Solución
404 Not Found	La ruta escrita no coincide con el endpoint.	Revisar /api/productos y el uso de @RequestMapping.
405 Method Not Allowed	Se usó un método HTTP incorrecto.	Verificar si el endpoint espera GET, POST, PUT, PATCH o DELETE.
400 Bad Request	El JSON no cumple con las validaciones o tiene tipos incorrectos.	Revisar nombres de campos y valores enviados.
415 Unsupported Media Type	No se envió Content-Type: application/json.	Configurar el header en Postman o curl.
No aparece @Valid	Falta importar jakarta.validation.Valid.	Revisar imports y dependencia Validation.
La aplicación no inicia	Puerto 8080 ocupado.	Cerrar otra aplicación o cambiar server.port.
Clase no detectada por Spring	Paquete creado fuera del paquete base.	Mover controller/service debajo de com.utp.semana4.

28. Ejercicios complementarios

Ejercicio 1: búsqueda por texto

Implemente un endpoint que permita buscar productos cuyo nombre contenga un texto parcial.

```
GET /api/productos/buscar?texto=lap
```

Debe devolver productos como Laptop Lenovo o Laptop Lenovo ThinkPad.

Ejercicio 2: disminuir stock

Implemente un endpoint que permita disminuir el stock de un producto.

```
PATCH /api/productos/{id}/disminuir-stock
```

Regla de negocio: no se debe permitir que el stock quede negativo. Si la cantidad a disminuir es mayor que el stock actual, responder con 400 Bad Request.

Ejercicio 3: documentación manual de API

Elabore una tabla documentando todos los endpoints implementados. Incluya método, URL, body esperado, código de respuesta y descripción. Esta práctica prepara al estudiante para documentar APIs en proyectos reales.

29. Preguntas de reflexión

1. ¿Por qué GET no debe usarse para crear, actualizar o eliminar datos?
2. ¿Qué diferencia existe entre POST y PUT?
3. ¿Qué diferencia existe entre PUT y PATCH?
4. ¿Por qué es útil ResponseEntity en una API REST?
5. ¿Qué función cumple @RequestBody?
6. ¿Qué diferencia hay entre @PathVariable y @RequestParam?
7. ¿Por qué conviene usar DTOs en lugar de recibir directamente cualquier objeto?
8. ¿Qué significa devolver un error 400?
9. ¿Qué significa devolver un error 404?
10. ¿Qué ventaja tiene centralizar errores con @RestControllerAdvice?

30. Cierre de la práctica

En esta guía el estudiante implementó una API REST completa con Spring Boot, aplicando endpoints, métodos HTTP, validaciones, respuestas JSON, códigos de estado y pruebas con herramientas externas. Esta práctica consolida la Unidad 1 y deja la base lista para integrar TDD, controladores, servicios y, posteriormente, persistencia con JPA, Hibernate y MySQL.

31. Referencias utilizadas

- Spring Boot Documentation. System Requirements.
- Spring Framework Documentation. Annotated Controllers and Mapping Requests.
- Spring Guides. Building REST services with Spring.